Popravni test iz predmeta
Algoritama i struktura podataka

Drugi praktični test održan 12.01.2024. Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. 0Ukupan broj bodova na testu je 30 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP_Test2_GrupaG_IT1g2021**
8. Ne postoji minimum bodova koji se mogu osvojiti iz svakog pojedinačnog zadatka
9. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova**
10. Ukoliko je u napomeni za rešavanja navedeno da je potrebno **koristiti metode date u šablonu**, da bi se ostvario maksimalan broj bodova **neophodno je njihovo korišćenje**. Tačno rešenje koje ne koristi ove algoritme se boduje brojem bodova manjim od maksimalnog.
11. Ukoliko za algoritam (rešenje zadatka) nije obezbeđen poziv odgovarajuće metode u *Main*-u kojom se algoritam testira, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova**

Pre početka testa napišite ime i prezima i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Konverzija liste u matricu susedstva*

Napisati algoritam koji će vršiti direktnu konverziju liste susedstva u matricu susedstva. Algoritam kao ulazni parametar treba da prihvati dinamički niz čvorova, a treba da rezultuje kreiranom matricom.

a) Napisati algoritam.
b) U *Main* metodi kreirati proizvoljan graf. Obezbediti način za testiranje i proveru nove funkcionalnosti, odnosno poziv ***convertListToMatrix*** metode.

Dodatne napomene za rešavanje zadatka:

- U template-u je zakomentarisana metoda ***convertListToMatrix*** koja treba da sadrži implementaciju konverzije liste u matricu susedstva.

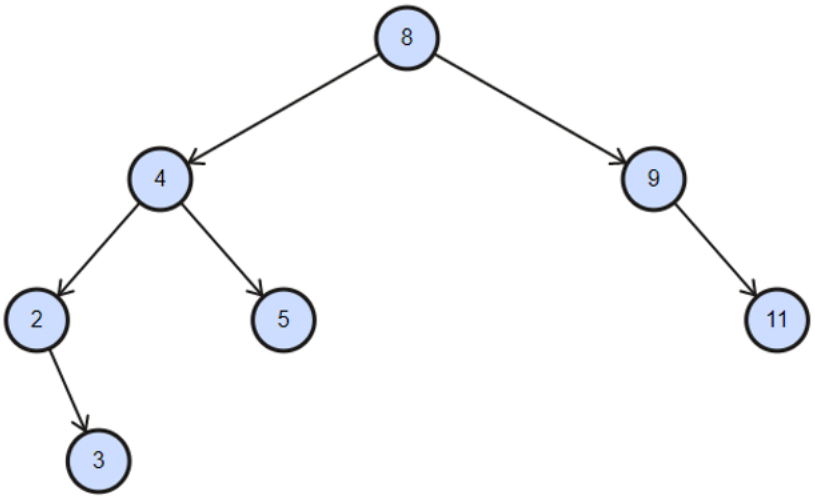
Zadatak 2 – Ispisivanje svih parnih vrednosti čvorova

U okviru standardne strukture klase ***AVL*** napisati implementaciju metode koja će omogućiti pronalaženje i ispisivanje čvorova sa parnom vrednošću.

Dodatne napomene za rešavanje zadatka:

- U okviru klase ***AVL*** kreirati javnu metodu sa potpisom ***public void printEvenNodes()***.
- Iz prethodno kreirane javne metode treba da se poziva privatna metoda sa potpisom ***private Node printEvenNodesAVL(Node curr)***
- Privatna metoda treba da sadrži implementaciju algoritma kojim će se omogućiti prolazak kroz stablo od korena ka listovima i njihovo ispisivanje ukoliko im je parna vrednost.
- Listom se klasifikuje svaki čvor koji nema nijedno dete.
- Može se kreirati pomoćna promenljiva na nivou klase AVL koja će pomoći u prebrojavanju.
- Algoritam **OBAVEZNO** pisati u okviru klase ***AVL.***

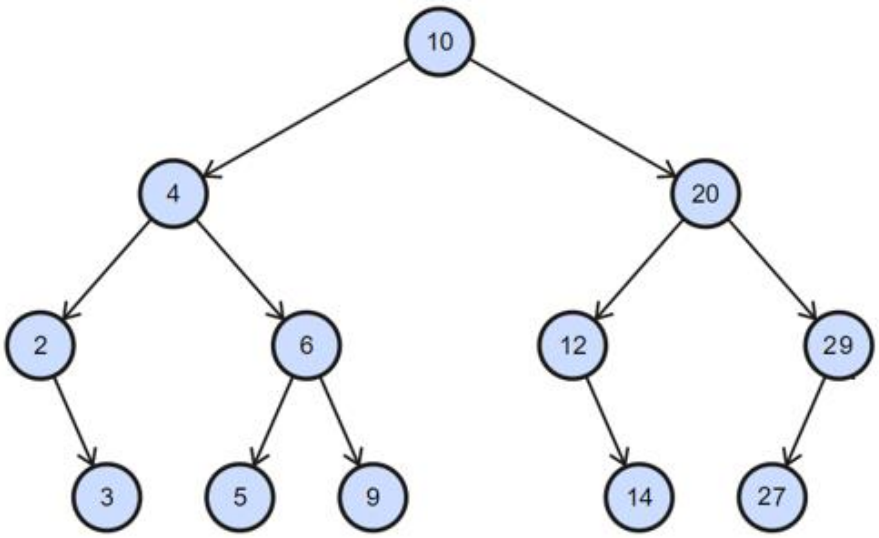
Primeri:



Slika 1 - Primer AVL-a

Input 1: *AVL sa slike 1*

Output 1: *8, 4, 2*



Slika 2 - Primer AVL-a

Input 2: *AVL sa slike 2*

Output 2: *10, 4, 2, 6, 20, 12, 14*

Zadatak 3 - *Ukrštanje sekvenci sa očuvanjem podsekvence*

Napisati algoritam koji ukršta dve sekvence, ukoliko je to moguće. Prilikom ukrštanja neophodno je u obe sekvence očuvati određenu podsekvencu, pa je njenom pozicijom određena pozicija ukrštanja. Uslov za ukrštanje sekvenci je da je estimirana sličnost sekvenci koje se ukrštaju manja od 0,9. U konzoli ispisati ukrštene sekvence ili poruku da ukrštanje nije moguće izvesti ukoliko je estimirana sličnost sekvenci koje se ukrštaju veća od 0,9. Algoritam implementirati uz pretpostavku da data subsekventa (npr. CGT iz 1. primera) sigurno postoji u sekvencama koje su date i pojavljuje se samo jednom u njima.

a) Napisati algoritam.
b) U *Main* metodi obezbediti testiranje, pozivom metode sa parametrima koji su dati u primerima.

Dodatne napomene za rešavanje zadatka:

- Koristiti *search* metodu *KarpRabinAlgorithm* klase i klase *MinHash* i *Jaccard,* koje su date u šablonu.
- Implementaciju pisati u okviru zakomentarisane metode ***public static void intersectSequencesAndKeepSubsequence (string str1, string str2, string substr).***
- Prilikom kreiranja MinHash objekta, za vrednosti parametara ***universeSize*** i ***numHashFunctions*** prosleđivati vrednosti ***12*** i ***4***
- Parametar ***q*** funkcije *search* postaviti na ***substr.Length+1***
- **Dozvoljeno** je izmeniti povratan tip metode *search* (npr. umesto *void* imati *List<int>*)

Napomene koje mogu biti od pomoći prilikom rešavanja zadatka:

- Kako biste string konvertovali u niz karaktera sa kojim se može vršiti manipulacija, koristiti metodu ***ToCharArray()*** po principu: ***str.ToCharArray()***
- Kako biste od niza karaktera dobili string, koristiti ***new string(char[] arrayOfChars)*** po principu: ***new string(charArray)***
- Karakteri se ne moraju eksplicitno konvertovati u celobrojni tip prilikom ubacivanja u *List<uint>* strukturu

Primer:

- Input: “ABH**CGT**MW”, “DHMW**CGT**R”, “CGT”
- Output: Ukrštene sekvence su: ABHCGT**R** i DHMWCGT**MW**

  (Sekvence ABHCGTMW i DHMWCGTR imaju estimiranu sličnost manju od 0,9. Mesto ukrštanja određeno je krajnjom pozicijom subsekvence CGT)

- Input: “ABH**CGT**MW”, “ABHW**CGT**M”, “CGT”
- Output: Nije moguće izvršiti ukrštanje.

  (Sličnost sekvenci ABHCGTMW i ABHWCGTM je preko 0,9)